

ViewGroups no Android

No Android, um ViewGroup é um tipo especial de View que pode conter outras Views dentro dele.

Ele funciona como um container que organiza os elementos da interface gráfica (botões, textos, imagens, etc.).

O ViewGroup define como os elementos filhos serão posicionados e dimensionados na tela.

Enquanto uma View representa um componente individual (ex.: TextView, Button), um ViewGroup organiza várias Views dentro dele.

Tipos de ViewGroup

LinearLayout

O LinearLayout organiza os elementos em linha única, podendo ser na vertical ou horizontal.

Vantagem: simples e fácil de usar.

Desvantagem: pode ficar ineficiente em layouts complexos.

Exemplo XML:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

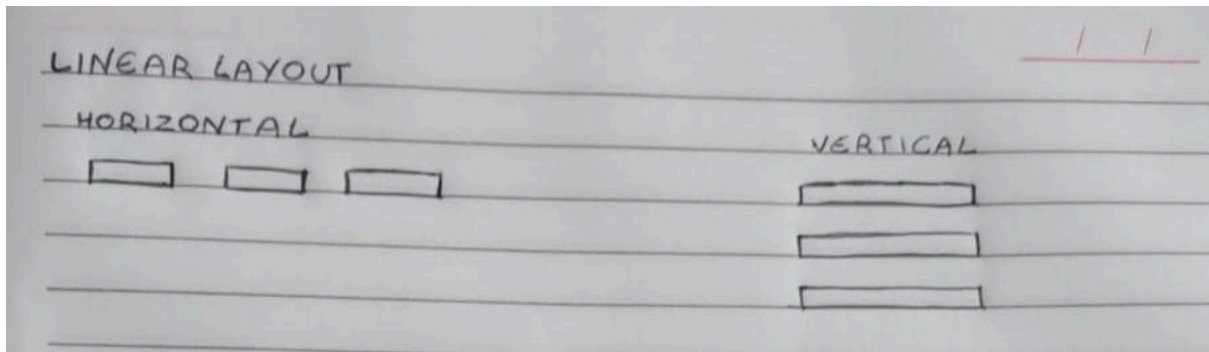
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Nome:" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Enviar" />

</LinearLayout>
```

Exemplo gráfico:



RelativeLayout

O RelativeLayout organiza elementos em relação a outros ou ao próprio container.

Vantagem: flexível para posicionar elementos.

Desvantagem: pode se tornar complexo com muitos componentes.

Exemplo XML:

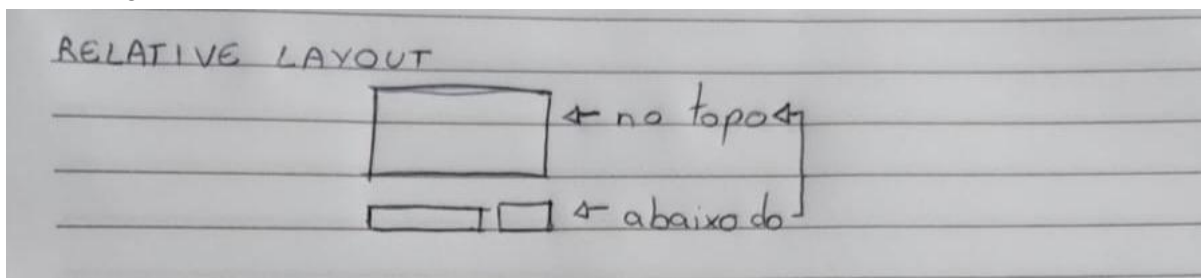
```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/titulo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Bem-vindo!" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/titulo"
        android:layout_alignParentEnd="true"
        android:text="OK" />

</RelativeLayout>
```

Exemplo gráfico:



FrameLayout

O FrameLayout é usado para sobrepor elementos, como uma pilha. Muito usado para fragments, imagens de fundo, sobreposições, etc.

Exemplo XML:

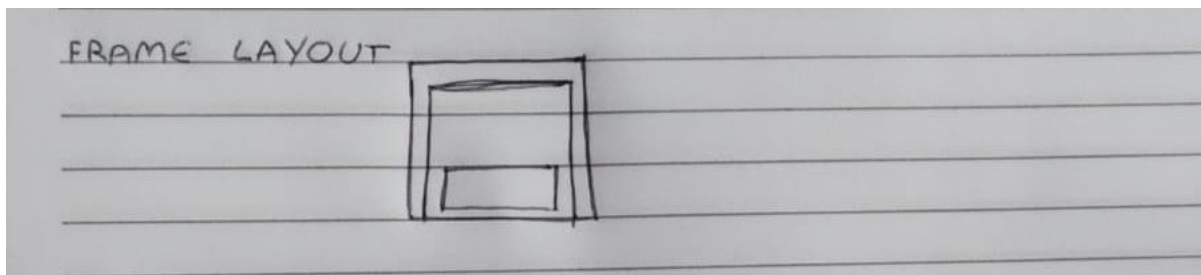
```
<FrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:src="@drawable/background" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto sobre a imagem"
        android:layout_gravity="center" />

</FrameLayout>
```

Exemplo gráfico:



ConstraintLayout

O ConstraintLayout é o mais poderoso e flexível.

Permite posicionar elementos com constraints (restrições) relativas a outros elementos ou ao container.

Vantagem: substitui layouts aninhados (mais performático).

Desvantagem: curva de aprendizado maior.

Exemplo XML:

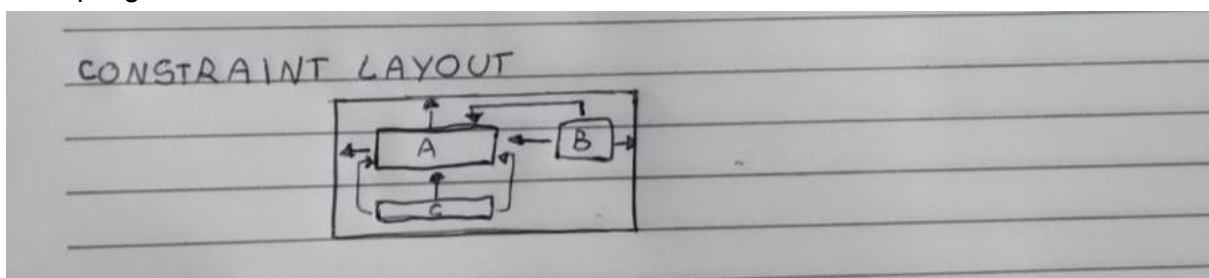
```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/titulo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/inputEmail"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/titulo"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:hint="E-mail" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Exemplo gráfico:



ScrollView

O ScrollView permite rolar o conteúdo verticalmente.

Só pode ter um filho direto, mas esse filho pode conter vários elementos dentro.

Exemplo XML:

```
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Item 1" />

        <!-- Imagine vários outros itens aqui -->

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Item 20" />

    </LinearLayout>
</ScrollView>
```

Exemplo gráfico:

